

EXPERIMENT 6

DECISION TREE

Aim:- To implement the Decision Tree algorithm for both binary and multi-class classification by learning feature-based decision rules through recursive data partitioning and to analyze its predictive performance.

Theory:

1. Introduction to Decision Tree

A **Decision Tree** is one of the most popular supervised machine learning algorithms used for both **classification and regression problems**. It is called a decision tree because its structure resembles a tree where decisions are made step by step.

In a decision tree, the dataset is divided into smaller groups based on certain conditions on the features. These conditions are arranged in a tree-like structure where each path from the root node to a leaf node represents a sequence of decisions that leads to a final prediction.

Because of its **simple structure and easy interpretation**, the decision tree algorithm is widely used in many applications such as medical diagnosis, customer purchase prediction, loan approval systems, and fraud detection.

2. Structure of a Decision Tree

A decision tree consists of several components that together form the structure of the model as shown the Figure 1.

2.1 Root Node

The root node is the topmost node of the decision tree. It represents the entire dataset and is the starting point of the decision-making process. The algorithm selects the most important feature at this node to split the dataset.

2.2 Internal Nodes

Internal nodes are also called decision nodes. These nodes represent conditions or tests applied to the dataset. Based on the result of the test, the data moves to the next branch.

2.3 Branches

Branches represent the possible outcomes of a test performed at a node. Each branch connects one node to another and shows the direction that the data follows depending on the attribute value.

2.4 Leaf Nodes

Leaf nodes are also called terminal nodes. These nodes represent the final decision or prediction made by the model. In classification problems, they represent class labels, while in regression problems they represent numerical values.

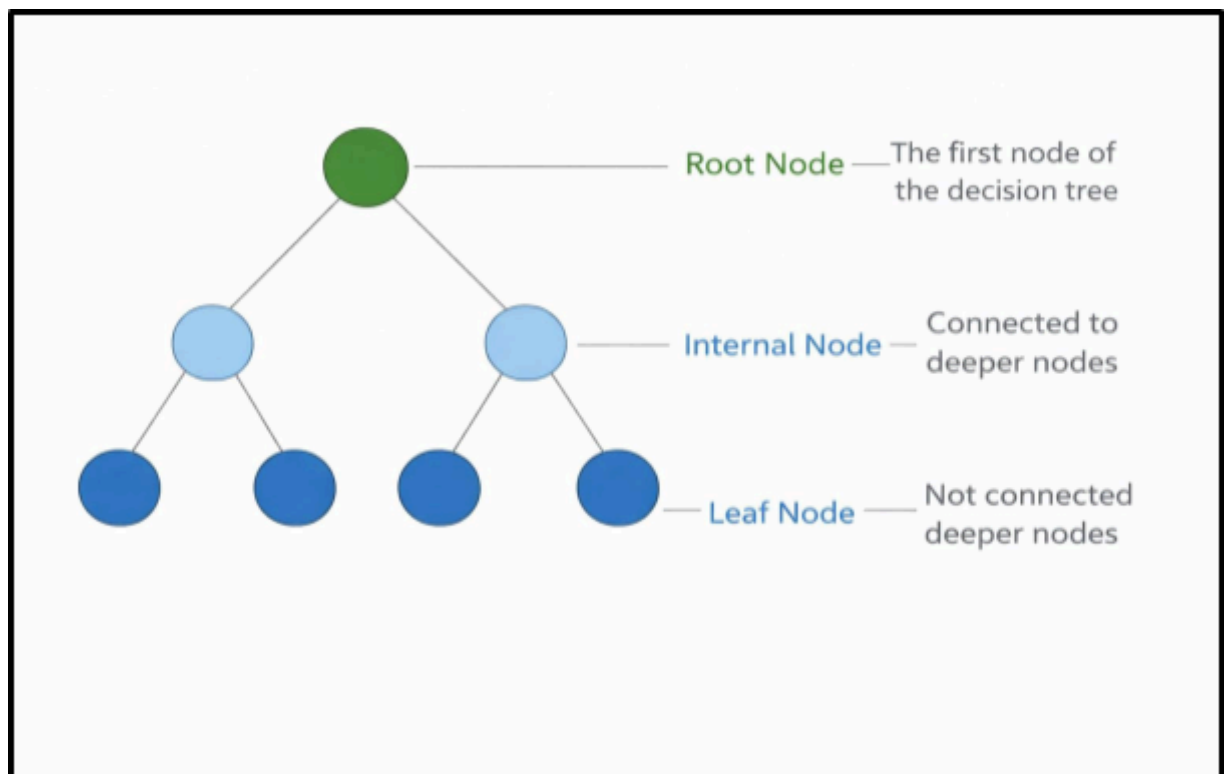


Figure 1: Basic structure of a decision tree

3. Decision Tree Learning Concept

The learning process of a decision tree revolves around one central principle: recursively partitioning the training data into progressively purer subsets with respect to the target variable. The goal is to create regions in the feature space where most or all samples belong to the same class (for classification) or have similar output values (for regression).

At the start of training, the entire dataset is placed at the root node. The algorithm then searches through all available features and all possible split points for each feature to find the split that best separates the data. Once a split is chosen, the data

is divided into subsets, and the same process is applied recursively to each subset. This continues until a stopping condition is met.

The concept of homogeneity is central to this process. A node is considered pure if all the samples it contains belong to the same class. As the tree grows deeper, each node should ideally contain a more homogeneous set of samples than its parent.

This recursive partitioning is what allows decision trees to model complex, non-linear relationships between features and the target variable without requiring explicit feature engineering. The recursive nature of this process means that the tree grows by repeatedly applying the same splitting logic to smaller and smaller subsets of the data. At each level of the tree, the algorithm makes the locally optimal choice of feature and split point, which is known as a greedy approach. While this greedy strategy is computationally efficient, it does not guarantee a globally optimal tree structure.

4. Splitting Criteria

To decide which feature should be used to split the dataset at each node, the decision tree algorithm uses certain mathematical measures known as **splitting criteria**. Two commonly used criteria are **Entropy (Information Gain)** and **Gini Index**.

4.1 Entropy

Entropy measures the **uncertainty or impurity** in a dataset. If all the data points belong to the same class, the entropy value becomes zero, which means the data is perfectly pure. If the data contains a mixture of different classes, the entropy value becomes higher.

Entropy is calculated using the following formula:

$$Entropy(S) = - \sum_i p_i \log_2(p_i)$$

where p_i represents the probability of each class in the dataset.

Using entropy, the algorithm calculates **Information Gain**, which measures how much uncertainty is reduced after splitting the dataset using a particular attribute. The attribute with the **highest information gain** is selected for splitting.

4.2 Gini Index

The **Gini Index** is another measure used to evaluate the quality of a split. It represents the probability of incorrectly classifying a randomly chosen element from the dataset.

The formula for Gini Index is:

$$Gini = 1 - \sum p_i^2$$

A **lower Gini value** indicates that the node is purer and contains mostly samples from a single class.

5. Tree Construction Process

The construction of a decision tree follows a systematic, top-down procedure. Beginning with the entire dataset at the root, the tree is built by repeatedly applying splitting rules to create child nodes, expanding level by level until one or more stopping conditions are satisfied. The general process unfolds as follows:

1. The entire training dataset is placed at the root node.
2. The algorithm evaluates every feature and every possible split threshold for that feature.
3. For each candidate split, the splitting criterion (Entropy or Gini Index) is computed.
4. The feature and split point that produce the best criterion value are selected.
5. The dataset is divided into subsets based on the chosen split, and child nodes are created.
6. Steps 2 through 5 are applied recursively to each child node using only the samples that reached that node.
7. The process continues until a stopping condition is met, at which point the node becomes a leaf.

A decision tree stops growing when certain stopping conditions are satisfied. The growth of the tree terminates if all samples within a node belong to the same class, if the number of samples in the node becomes too small to allow further meaningful splitting, or if a predefined maximum depth of the tree is reached. The splitting process may also stop when further partitioning does not improve the chosen decision criterion, in which case the node is designated as a leaf node and assigned a class label. It is important to note that the decision tree construction algorithm follows a greedy approach, meaning that at each node it selects the best possible split based only on the current data without reconsidering previous decisions. As a result, the final tree structure may not always represent the globally optimal solution,

which is why techniques such as pruning and ensemble methods are often employed to improve model performance.

6. Pruning Techniques

One of the most significant challenges with decision trees is overfitting. When a tree is allowed to grow without restrictions, it tends to learn the training data too precisely, including noise and outliers, resulting in a highly complex model that performs poorly on new, unseen data. Pruning is the process of simplifying a decision tree to reduce overfitting and improve its generalization capability.

Pre-Pruning (Early Stopping)

Pre-pruning, also known as early stopping, prevents the tree from growing beyond a certain point during the construction phase itself. Instead of building the full tree and then simplifying it, the algorithm stops creating new branches when certain conditions are met. Common pre-pruning strategies include:

- Setting a maximum depth limit on the tree, preventing it from growing too deep.
- Requiring a minimum number of samples before a node can be split further.
- Requiring that any split must improve the splitting criterion by at least a minimum amount.

Pre-pruning is computationally efficient because it avoids building parts of the tree that will later be removed. However, it risks stopping too early and producing an underfitted model that misses important patterns in the data.

Post-Pruning

Post-pruning is applied after the tree has been fully grown. The algorithm first builds the complete tree and then works backwards, evaluating whether removing certain subtrees or replacing them with leaf nodes improves the model's performance on a validation dataset. The most commonly used post-pruning method is Cost-Complexity Pruning (also called Reduced-Error Pruning or Weakest Link Pruning).

In Cost-Complexity Pruning, a complexity parameter (often called α) is introduced to penalize large tree sizes. By tuning this parameter, one can control the trade-off between tree size and accuracy. Larger values of α produce smaller, simpler trees, while smaller values allow more complex trees. Crossvalidation is typically used to select the optimal value of α .

Post-pruning generally produces better results than pre-pruning because it makes decisions based on the full picture of the tree, but it is more computationally demanding.

7. Decision Tree Algorithm

Step 1: Start with entire dataset at root node

Step 2: If all samples belong to same class, Create leaf node with that class label, **STOP**

Step 3: If no features remaining, Create leaf node with majority class, and **STOP**

Step 4: Select the best feature to split on:

Using Gini Impurity:

- $Gini(node) = 1 - \sum(p_i)^2$
- Where p_i = proportion of class i in the node
- For each feature and each possible split value:
 - > Calculate weighted Gini of child nodes
 - > $Gini_split = (n_left/n_total) \times Gini_left + (n_right/n_total) \times Gini_right$
- Select split with LOWEST Gini

Using Information Gain (Entropy):

- $Entropy(node) = -\sum(p_i \times \log_2(p_i))$
- $InformationGain = Entropy(parent) - Weighted_Entropy(children)$
- Select split with HIGHEST Information Gain

Step 5: Create child nodes based on the split condition.

Step 6: Recursively apply Steps 2-5 to each child node.

Step 7: For prediction:

- Start at root node.
- At each node, check the split condition.
- Move to left or right child based on feature value.
- Continue until reaching a leaf node.
- Return the class label of the leaf.

8. Merits of Decision Trees

Decision trees offer several important advantages that explain their widespread use across both academic research and real-world applications.

Interpretability and Transparency: Decision trees are among the most interpretable machine learning models. Their visual, hierarchical structure allows users, stakeholders, and domain experts to follow the exact reasoning path that led to any given prediction. This transparency is especially valuable in regulated industries such as healthcare and finance, where model decisions must be explainable.

Minimal Data Preprocessing: Unlike many other machine learning algorithms, decision trees do not require normalization, standardization, or scaling of input

features. They can handle raw numerical and categorical data directly, significantly reducing the amount of preprocessing work required before training.

Ability to Handle Mixed Feature Types: Decision trees can naturally accommodate datasets containing both numerical features (such as age or income) and categorical features (such as gender or occupation) without requiring any special encoding or transformation. **Automatic Feature Selection:** During the tree construction process, the algorithm automatically identifies and uses the most informative features at each node. Features that are not selected at any node do not influence the model, effectively performing built-in feature selection.

Non-Linear Relationship Modeling: Decision trees can capture complex, non-linear relationships between input features and the target variable. By partitioning the feature space into rectangular regions, they can approximate any decision boundary regardless of its shape.

Fast Prediction Time: Once a decision tree is trained, making predictions requires only traversing a single path from root to leaf, which is computationally very fast even for large trees.

9. Demerits of Decision Trees

Despite their many advantages, decision trees have several notable limitations that must be considered when selecting them for a given problem.

Prone to Overfitting: One of the most well-known weaknesses of decision trees is their tendency to overfit the training data, particularly when the tree is allowed to grow deep without restriction. An overfitted tree memorizes the training data, including noise and irrelevant patterns, resulting in poor performance on new, unseen data.

High Variance and Instability: Decision trees are highly sensitive to small changes in the training data. A slight modification in the dataset can lead to an entirely different tree structure. This instability, known as high variance, makes individual decision trees unreliable as standalone models for high-stakes applications.

Greedy Learning and Suboptimal Splits: The tree construction algorithm uses a greedy approach, making the locally best split at each node without considering the global structure of the tree. This can lead to suboptimal splits that miss better overall tree configurations.

Difficulty with Complex Relationships: While decision trees can model non-linear relationships, they struggle to capture certain complex patterns such as smooth curves or interactions that require many small splits. Such patterns often require very deep trees, increasing the risk of overfitting.

Biased Towards Features with More Levels: Decision trees tend to favor features with more possible split points (e.g., continuous or high-cardinality categorical features) over features with fewer levels. This bias can lead to the selection of less informative features in some situations.

Lower Accuracy Compared to Ensembles: A single decision tree typically achieves lower predictive accuracy than more advanced ensemble methods such as Random Forests, Gradient Boosted Trees, or XGBoost, which combine many trees to produce more robust and accurate predictions.

10. Procedure:

Part 1: Binary Classification

The objective of this experiment is to classify applicants into two categories—loan approved and loan not approved—based on a set of applicant attributes. The input to the model consists of multiple independent variables representing financial and personal features of the applicants, while the output is a binary dependent variable indicating whether the loan is approved or not.

Step 1: Import required libraries: pandas, matplotlib, Seaborn.

Step 2: Load '**Loan_dataset.csv**'. The Loan Approval Prediction dataset contains 4,269 rows and 13 columns. It includes applicant details such as income, loan amount, credit history, and employment status, with the target column indicating whether the loan is approved or not.

Step 3: Perform data analysis using `head()`, `tail()`, `info()`, `describe()`, and `isnull().sum()`.

Step 4: Define Features of X and Y:

- Features X: `loan_id`, `no_of_dependents`, `education`, `self_employed`, `income_annum`, `loan_amount`, `loan_term`, `cibil_score`, `residential_assets_value`, `commercial_assets_value`, `luxury_assets_value`, `bank_asset_value`
- Y: `loan_status`

Step 5: Split the Dataset into 80% Training Set and 20% Testing Sets.

Step 6: Train the Decision Tree Classifier: In this step, the Decision Tree Classifier is initialized and configured by setting important parameters such as the splitting criteria and the maximum depth (`max_depth`) of the tree. After defining these parameters, the classifier is trained using the training dataset, allowing it to learn decision rules from the data and build a tree structure for accurate prediction.

Step 7: Make Predictions on Test Data: Using the trained Decision Tree model, predictions are made on the test dataset by providing its feature values as input. These predicted outcomes are then compared with the actual values to evaluate the performance of the model.

Step 8: Evaluate Model Performance: The Decision Tree model is evaluated using the accuracy score to measure overall correctness, the confusion matrix to display true and false predictions, and the classification report to summarize precision, recall, and F1-score for each class. These metrics together help assess the model's performance.

Step 9: Plot: To visualize the model's performance, a confusion matrix heatmap is plotted.

Part 2: Multi-class Classification

The objective of this experiment is to classify individuals into different obesity categories, such as normal weight, overweight, or various obesity levels, based on a set of personal and lifestyle attributes. The input to the model consists of multiple independent variables, including demographic information (e.g., age, gender, height, weight), dietary habits, physical activity, and other behavioral factors, while the output is a categorical dependent variable indicating the obesity level of each individual.

Step 1: Import required libraries: pandas, matplotlib, seaborn.

Step 2: Load '**Obesity_Dataset.csv**'. The obesity dataset contains 2,111 records and 17 columns, representing individuals with various lifestyle, dietary, and physical attributes. It is used to classify obesity levels, where multiple personal and behavioral features act as inputs and the obesity category is the target variable.

Step 3: Define Feature X and Y:

- Features X (Independent Variables): Gender, Age, Height, Weight, family_history_with_overweight, FAVC, FCVC, NCP, CAEC, SMOKE, CH2O, SCC, FAF, TUE, CALC, MTRANS
- Target Variable Y: NObeyesdad (represents the obesity category/class of an individual)

Step 4: Split the Dataset into 80% Training Set and 20% Testing Sets.

Step 5: Train the Decision Tree Classifier: In this step, the Decision Tree Classifier is initialized and configured by setting important parameters such as the splitting criteria and the maximum depth (max_depth) of the tree. After defining these parameters, the classifier is trained using the training dataset, allowing it to learn decision rules from the data and build a tree structure for accurate prediction.

Step 6: Make Predictions on Test Data: Using the trained Decision Tree model, predictions are made on the test dataset by providing its feature values as input. These predicted outcomes are then compared with the actual values to evaluate the performance of the model.

Step 7: Evaluate Model Performance: The Decision Tree model is evaluated using the accuracy score to measure overall correctness, the confusion matrix to display true and false predictions, and the classification report to summarize precision, recall, and F1-score for each class. These metrics together help assess the model's performance.

Step 8: Plot: To visualize the model's performance, a confusion matrix heatmap is plotted.

Decision Trees - Practical Implementation

Option A: Binary Classification (Loan Approval Dataset)

Step 1: Importing Libraries

Import necessary libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
import matplotlib.pyplot as plt
import seaborn as sns
print("Libraries imported successfully!")
```

Output:

```
Libraries imported successfully!
```

Step 2: Initial Data Loading & Exploration

Load the dataset

```
data = pd.read_csv("loan_approval_dataset.csv")
print("Dataset loaded successfully")
```

Output:

```
Dataset loaded successfully
```

Check the shape of the dataset

```
print("Dataset Shape:", data.shape)
```

Output:

```
Dataset Shape: (4269, 13)
```

Fix column spacing

```
data.columns = data.columns.str.strip()

# Print columns in vertical form
print("\n\nColumns (Vertical List):")
for col in data.columns:
    print(col)
```

Output:

```
Columns (Vertical List):
loan_id
no_of_dependents
education
self_employed
income_annum
loan_amount
loan_term
cibil_score
residential_assets_value
commercial_assets_value
luxury_assets_value
bank_asset_value
loan_status
```

Display first 5 rows

```
print("First 5 rows of the dataset:")
data.head()
```

Output:

```
First 5 rows of the dataset:
```

Output:

	loa	no_of_depen	educatio	self_emp.	income_.	loan_am	loan_	cibil_s	residential_asset..	commercial_assets	luxury_assets.	bank_asset_	loan_st.
0	1	2	Graduate	No	9600000	2990000	12	778	2400000	17600000	22700000	8000000	Approved
1	2	0	Not Gra.		4100000	1220000		417	2700000	2200000	8800000	3300000	Rejected
2	3	3	Graduate	No	9100000	2970000	20	506	7100000	4500000	33300000	12800000	Rejected
3	4	3	Graduate	No	8200000	3070000		467	18200000	3300000	23300000	7900000	Rejected
4	5	5	Not Gra.	Yes	9800000	2420000	20	382	12400000	8200000	29400000	5000000	Rejected

Display last 5 rows

```
print("Last 5 rows of the dataset:")
data.tail()
```

Output:

```
Last 5 rows of the dataset:
```

Output:

	loa	no_of_depen	educatio	self_emp.	income_.	loan_am	loan_	cibil_s	residential_asset..	commercial_assets	luxury_assets.	bank_asset_	loan_st.
42	4265	5	Graduate	Yes	1000000	2300000	12	317	2800000	500000	3300000	800000	Rejected
42		0	Not Gra.		3300000	1130000		559	4200000	2900000	11000000	1900000	Approved
42	4267	2	Not Gra.	No	6500000	2390000	18	457	1200000	12400000	18100000	7300000	Rejected
42		1	Not Gra.		4100000	1280000		780	8200000	700000	14100000	5800000	Approved
42	4269	1	Graduate	No	9200000	2970000	10	607	17800000	11800000	35700000	12000000	Approved

Step 3: Data Preprocessing & Information

Target Column Encoding

```
data.columns = data.columns.str.strip()
data['loan_status'] = data['loan_status'].str.strip().map({
    'Approved': 1,
    'Rejected': 0
})

print("Target column loan_status encoded successfully")
data.head()
```

Output:

Target column loan_status encoded successfully

Output:

	loa	no_of_depen	educatio	self_emp.	income_	loan_am	loan_	cibil_s	residential_asset..	commercial_assets	luxury_assets.	bank_asset_	loan_st.
0	1	2	Graduate	No	9600000	2990000	12	778	2400000	17600000	22700000	8000000	1
1	2	0	Not Gra.	Yes	4100000	12200000	8	417	2700000	2200000	8800000	3300000	0
2	3	3	Graduate	No	9100000	2970000	20	506	7100000	4500000	33300000	12800000	0
3	4	3	Graduate	No	8200000	30700000	8	467	18200000	3300000	23300000	7900000	0
4	5	5	Not Gra.	Yes	9800000	2420000	20	382	12400000	8200000	29400000	5000000	0

Data Information

```
print("All Required Information Related To Data: ")
data.info()
```

Output:

```
All Required Information Related To Data:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4269 entries, 0 to 4268
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   loan_id                               4269 non-null   int64
1   no_of_dependents                       4269 non-null   int64
2   education                             4269 non-null   object
3   self_employed                         4269 non-null   object
4   income_annum                           4269 non-null   int64
5   loan_amount                           4269 non-null   int64
6   loan_term                             4269 non-null   int64
7   cibil_score                           4269 non-null   int64
8   residential_assets_value               4269 non-null   int64
9   commercial_assets_value               4269 non-null   int64
10  luxury_assets_value                   4269 non-null   int64
11  bank_asset_value                      4269 non-null   int64
12  loan_status                           4269 non-null   int64

dtypes: int64(11), object(2)
memory usage: 433.7+ KB
```

Check for missing (null) values in each column

```
print("Missing values in each column:")
print(data.isnull().sum())
```

Output:

```
Missing values in each column:
loan_id                0
no_of_dependents      0
education              0
self_employed         0
income_annum          0
loan_amount           0
loan_term             0
cibil_score           0
residential_assets_value 0
commercial_assets_value 0
luxury_assets_value   0
bank_asset_value      0
loan_status           0
dtype: int64
```

Step 4: Feature Engineering & Data Splitting

Define target column

```
target_col = "loan_status"
print("Target column selected:", target_col)

# Separate features and target
X = data.drop(columns=[target_col])
y = data[target_col]
print("Features (X) and target (y) separated successfully")

# Identify categorical columns
cat_cols = X.select_dtypes(include=['object']).columns.tolist()
print("Categorical columns identified:", cat_cols)

# Apply One-Hot Encoding to categorical columns
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
    ],
    remainder='passthrough'
)

# Transform feature data
X_transformed = preprocessor.fit_transform(X)
print("Categorical features one-hot encoded and dataset transformed successfully")
```

Output:

```
Target column selected: loan_status
Features (X) and target (y) separated successfully
Categorical columns identified: ['education', 'self_employed']
Categorical features one-hot encoded and dataset transformed successfully
```

Split the dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X_transformed,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Dataset successfully split into training and testing sets")
print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)
```

Output:

```
Dataset successfully split into training and testing sets
Training set size: (3415, 14)
Testing set size: (854, 14)
```

Step 5: Model Training & Evaluation (Gini Criterion)**# Initialize and train Decision Tree model using Gini impurity**

```
dt_model = DecisionTreeClassifier(criterion='gini', random_state=42)
dt_model.fit(X_train, y_train)
print("Decision Tree model trained successfully using Gini impurity")
```

Output:

```
Decision Tree model trained successfully using Gini impurity
```

Evaluate Decision Tree model (Gini impurity)

```
y_pred = dt_model.predict(X_test)
print("Accuracy (Gini, Full Tree):", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix (Gini, Full Tree):")
print(confusion_matrix(y_test, y_pred))
```

Output:

```
Accuracy (Gini, Full Tree): 0.9789227166276346
Confusion Matrix (Gini, Full Tree):
[[313  10]
 [ 8 523]]
```

Compute confusion matrix for Decision Tree model (Gini impurity)

```
print("Calculating confusion matrix (Gini)...")
cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix calculated successfully")
```

Output:

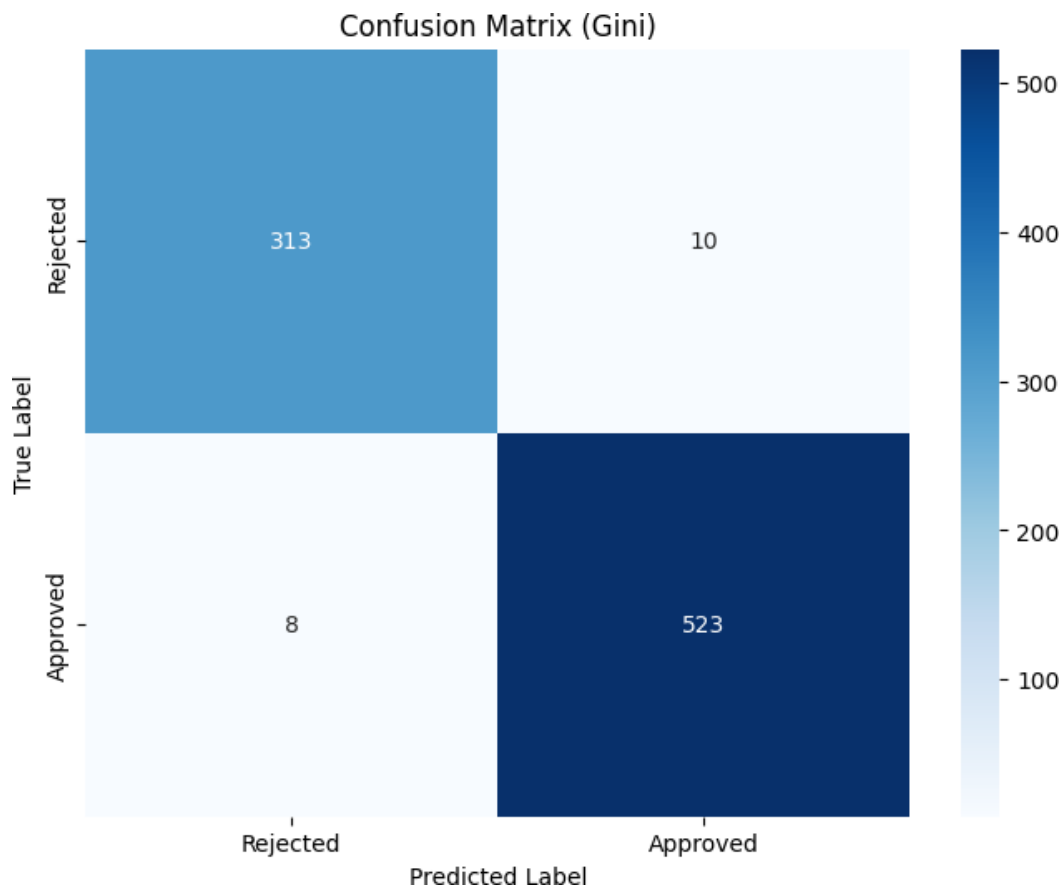
```
Calculating confusion matrix (Gini)...
Confusion matrix calculated successfully
```

Plot confusion matrix for Decision Tree model (Gini impurity)

```
print("Displaying confusion matrix plot (Gini)...")
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Rejected', 'Approved'],
            yticklabels=['Rejected', 'Approved'])
plt.title('Confusion Matrix (Gini)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
print("Confusion matrix plot displayed successfully")
```

Output:

Displaying confusion matrix plot (Gini)...Confusion matrix plot displayed successfully

**# Visualize the Gini Decision Tree**

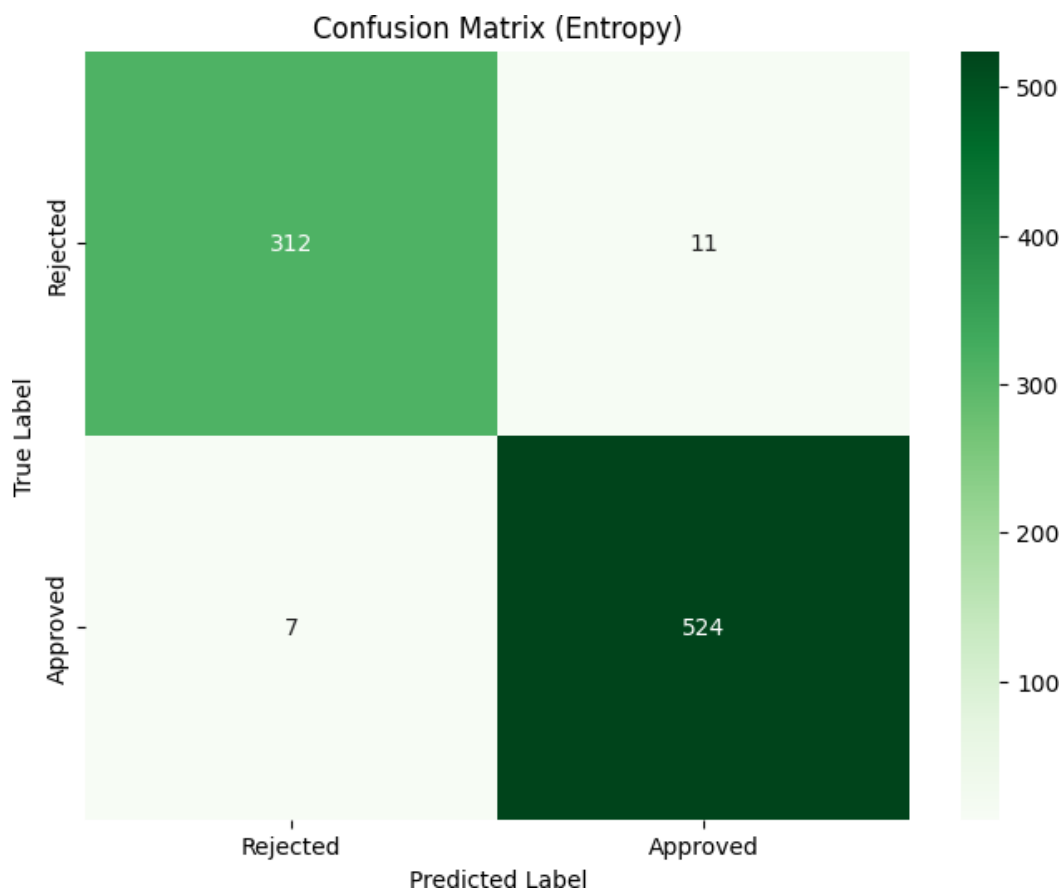
```
plt.figure(figsize=(20, 10))
plot_tree(dt_model, filled=True, rounded=True, label='root', impurity=False,
          proportion=False)
plt.title("Decision Tree (Gini)")
plt.show()
```


Plot confusion matrix for Decision Tree model (Entropy)

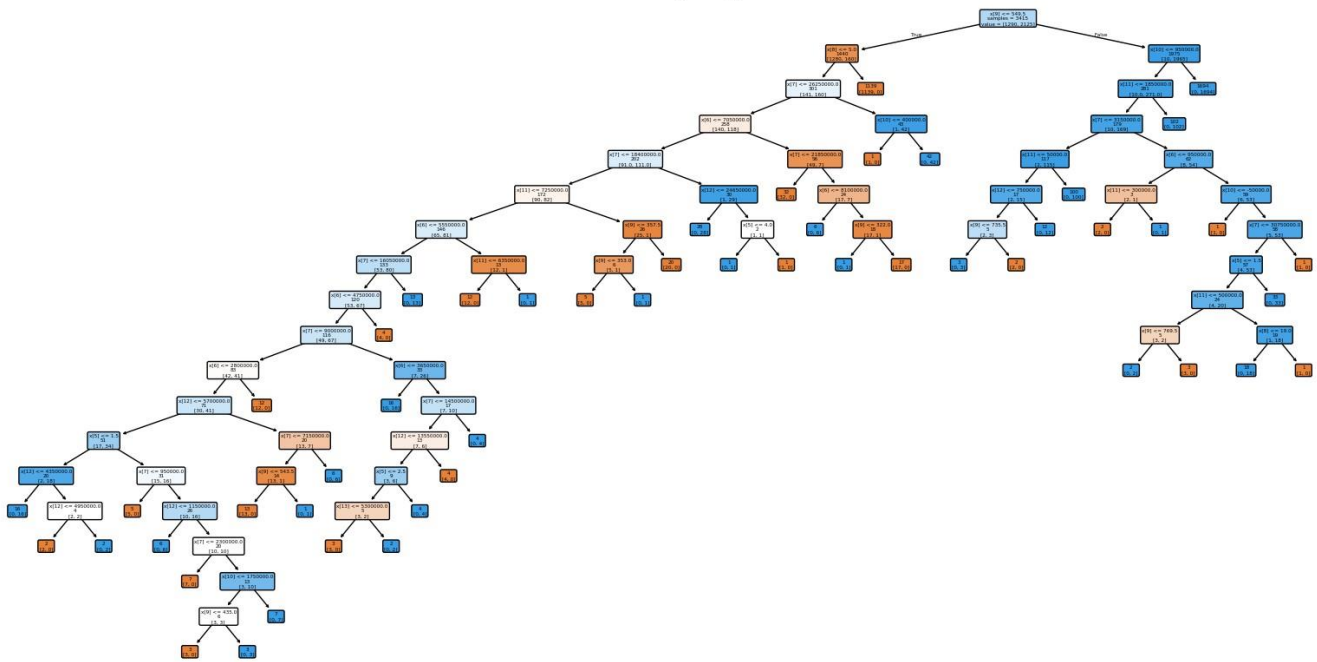
```
target_names = ['Rejected', 'Approved']
plt.figure(figsize=(8, 6))
sns.heatmap(cm_entropy, annot=True, fmt='d', cmap='Greens', xticklabels=target_names,
            yticklabels=target_names)
plt.title('Confusion Matrix (Entropy)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
print("Confusion matrix (Entropy) plot displayed successfully")
```

Output:

Displaying confusion matrix plot (Entropy)...Confusion matrix (Entropy) plot displayed successfully

**# Visualize the Entropy Decision Tree**

```
plt.figure(figsize=(20, 10))
plot_tree(dt_model_entropy, filled=True, rounded=True, label='root', impurity=False,
          proportion=False)
plt.title("Decision Tree (Entropy)")
plt.show()
```



Step 7: Decision Tree Depth Analysis (Gini)

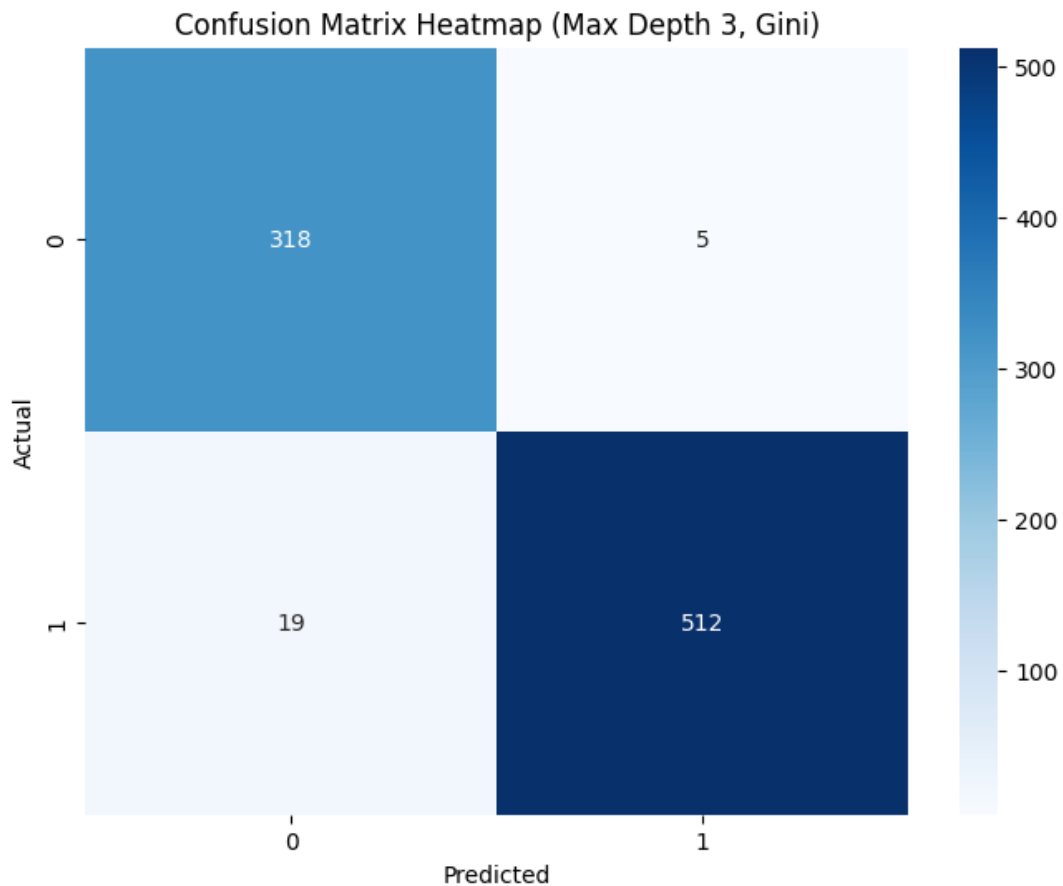
Train Decision Tree with max_depth=3 (Gini)

```
dt_gini_3 = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
dt_gini_3.fit(X_train, y_train)
y_pred_3 = dt_gini_3.predict(X_test)
accuracy_3 = accuracy_score(y_test, y_pred_3)
print(f"Accuracy (Max Depth 3, Gini): {accuracy_3:.4f}")
print("Confusion Matrix (Max Depth 3):")
print(confusion_matrix(y_test, y_pred_3))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_3), annot=True, fmt='d', cmap='Blues',
            xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 3, Gini)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output:

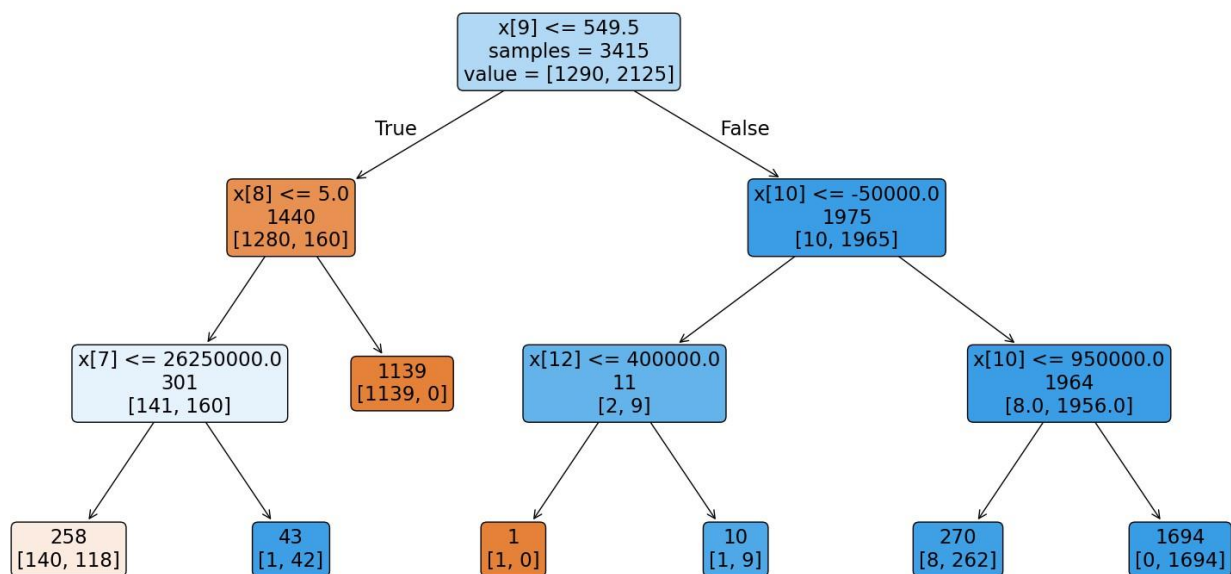
```
Accuracy (Max Depth 3, Gini): 0.9403
Confusion Matrix (Max Depth 3):
[[280  43]
 [  8 523]]
```



Plot the tree (Gini - Max Depth 3)

```
plt.figure(figsize=(20, 10))
plot_tree(dt_gini_3, filled=True, rounded=True, label='root', impurity=False,
          proportion=False)
plt.title("Decision Tree (Gini) - Max Depth 3")
plt.show()
```

Decision Tree (Gini) - Max Depth 3



Train Decision Tree with max_depth=5 (Gini)

```

dt_gini_5 = DecisionTreeClassifier(criterion='gini', max_depth=5, random_state=42)
dt_gini_5.fit(X_train, y_train)
y_pred_5 = dt_gini_5.predict(X_test)
accuracy_5 = accuracy_score(y_test, y_pred_5)
print(f"Accuracy (Max Depth 5, Gini): {accuracy_5:.4f}")
print("Confusion Matrix (Max Depth 5):")
print(confusion_matrix(y_test, y_pred_5))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_5), annot=True, fmt='d', cmap='Blues',
            xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 5, Gini)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

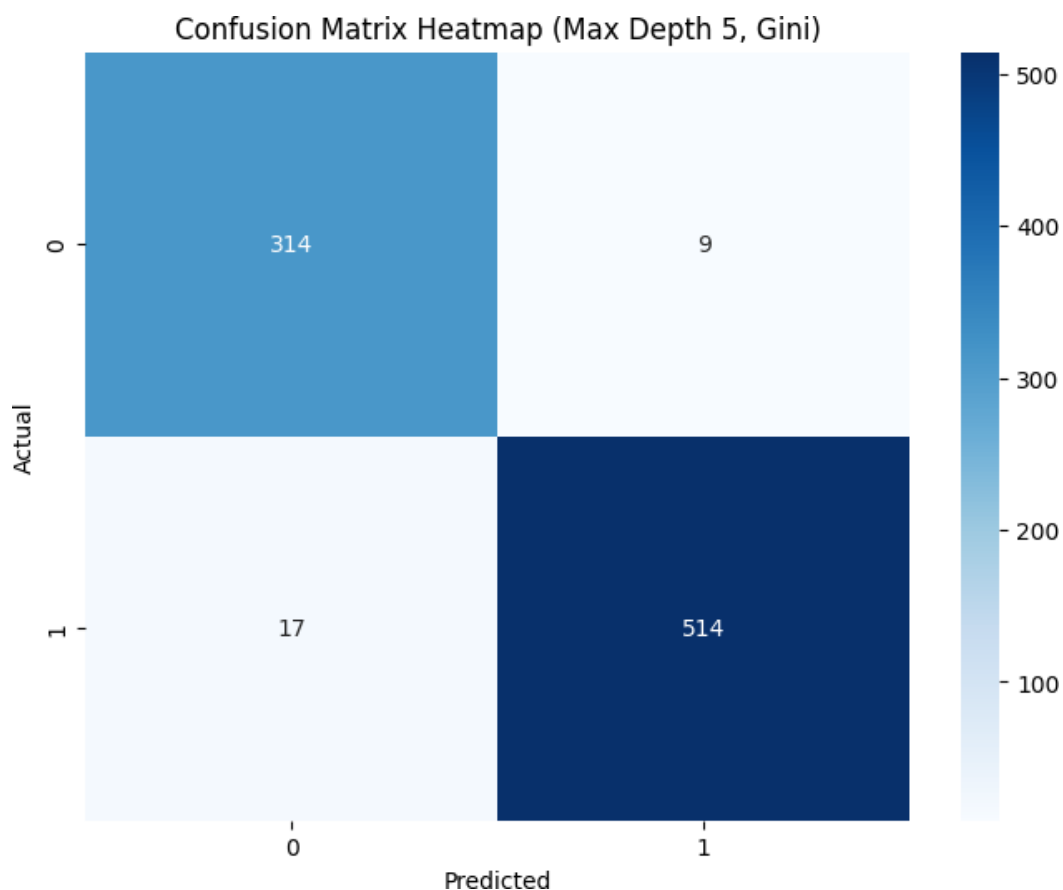
```

Output:

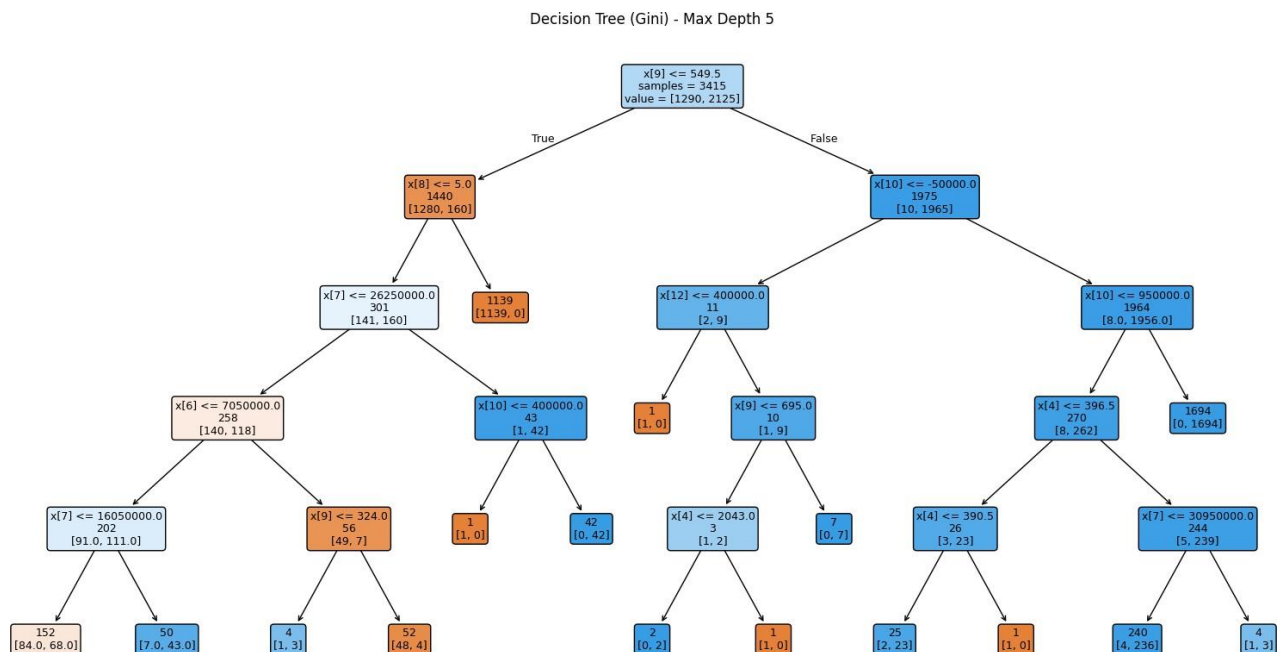
```

Accuracy (Max Depth 5, Gini): 0.9707
Confusion Matrix (Max Depth 5):
[[308 15]
 [ 10 521]]

```

**# Plot the tree (Gini - Max Depth 5)**

```
plt.figure(figsize=(20, 10))
plot_tree(dt_gini_5, filled=True, rounded=True, label='root', impurity=False,
          proportion=False)
plt.title("Decision Tree (Gini) - Max Depth 5")
plt.show()
```



Train Decision Tree with max_depth=7 (Gini)

```
dt_gini_7 = DecisionTreeClassifier(criterion='gini', max_depth=7, random_state=42)
dt_gini_7.fit(X_train, y_train)
y_pred_7 = dt_gini_7.predict(X_test)
accuracy_7 = accuracy_score(y_test, y_pred_7)
print(f"Accuracy (Max Depth 7, Gini): {accuracy_7:.4f}")
print("Confusion Matrix (Max Depth 7):")
print(confusion_matrix(y_test, y_pred_7))

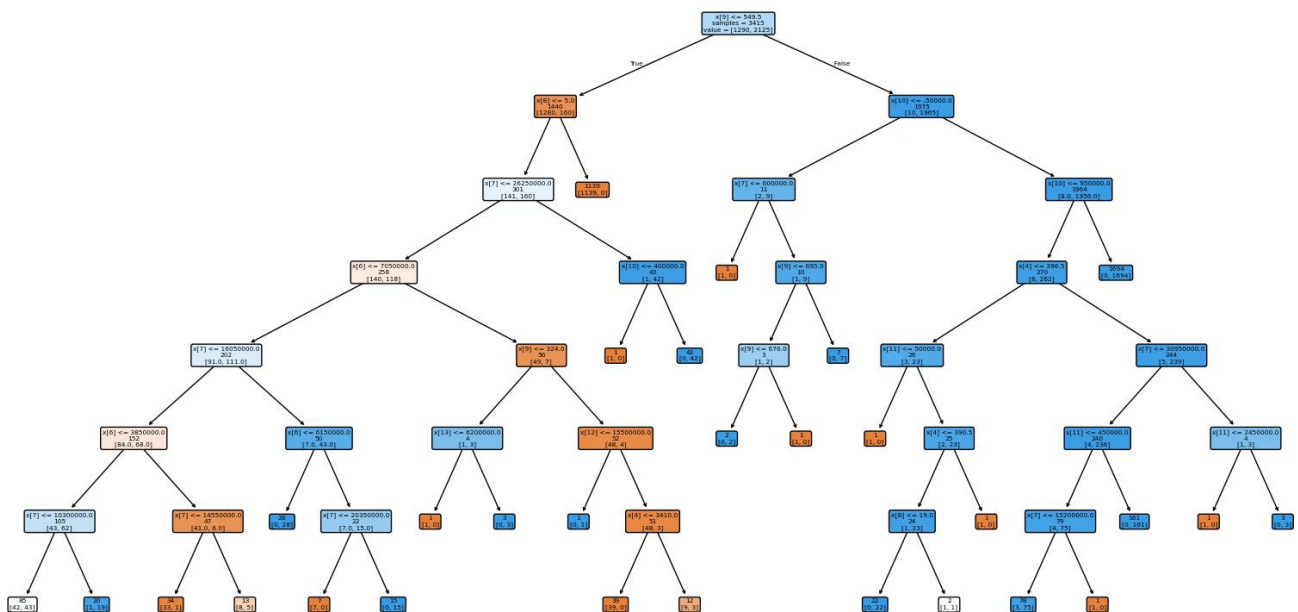
# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_7), annot=True, fmt='d', cmap='Blues',
            xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 7, Gini)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output:

```
Accuracy (Max Depth 7, Gini): 0.9766
Confusion Matrix (Max Depth 7):
[[312 11]
 [ 9 522]]
```



Decision Tree (Gini) - Max Depth 7



Step 8: Decision Tree Depth Analysis (Entropy)

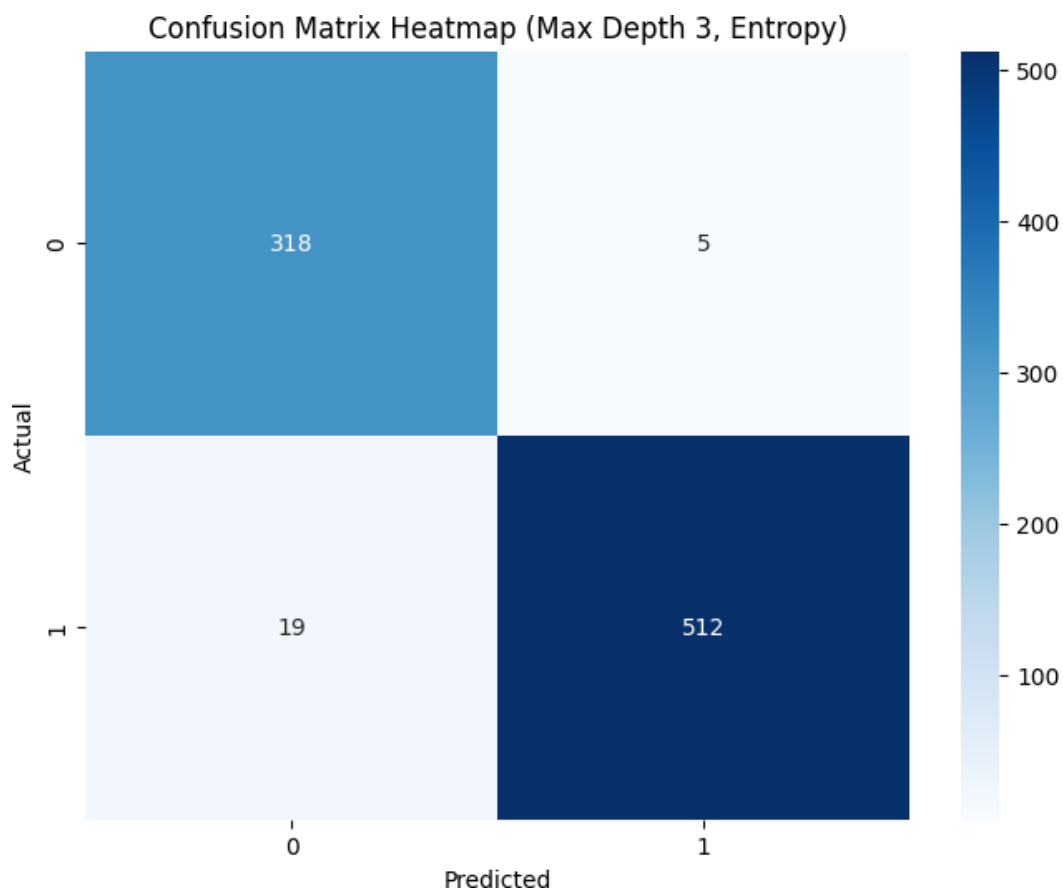
Train Decision Tree with max_depth=3 (Entropy)

```
dt_entropy_3 = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
dt_entropy_3.fit(X_train, y_train)
y_pred_3 = dt_entropy_3.predict(X_test)
accuracy_3 = accuracy_score(y_test, y_pred_3)
print(f"Accuracy (Max Depth 3, Entropy): {accuracy_3:.4f}")
print("Confusion Matrix (Max Depth 3):")
print(confusion_matrix(y_test, y_pred_3))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_3), annot=True, fmt='d', cmap='Blues',
            xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 3, Entropy)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output:

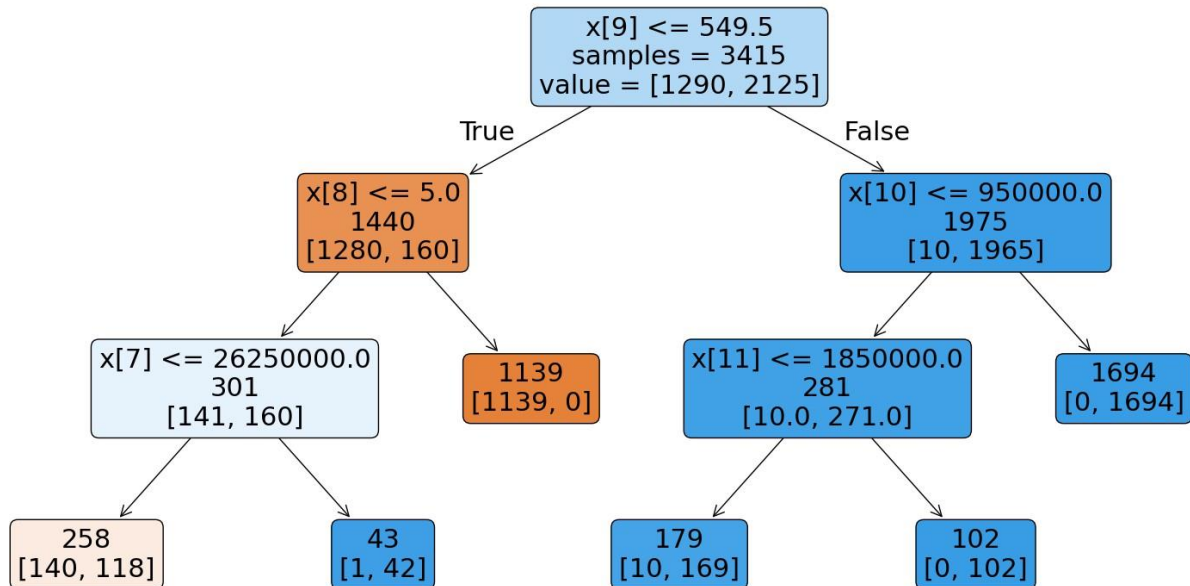
```
Accuracy (Max Depth 3, Entropy): 0.9333
Confusion Matrix (Max Depth 3):
[[275  48]
 [  9 522]]
```



Plot the tree (Entropy - Max Depth 3)

```
plt.figure(figsize=(20, 10))
plot_tree(dt_entropy_3, filled=True, rounded=True, label='root', impurity=False,
          proportion=False)
plt.title("Decision Tree (Entropy) - Max Depth 3")
plt.show()
```

Decision Tree (Entropy) - Max Depth 3



Train Decision Tree with max_depth=5 (Entropy)

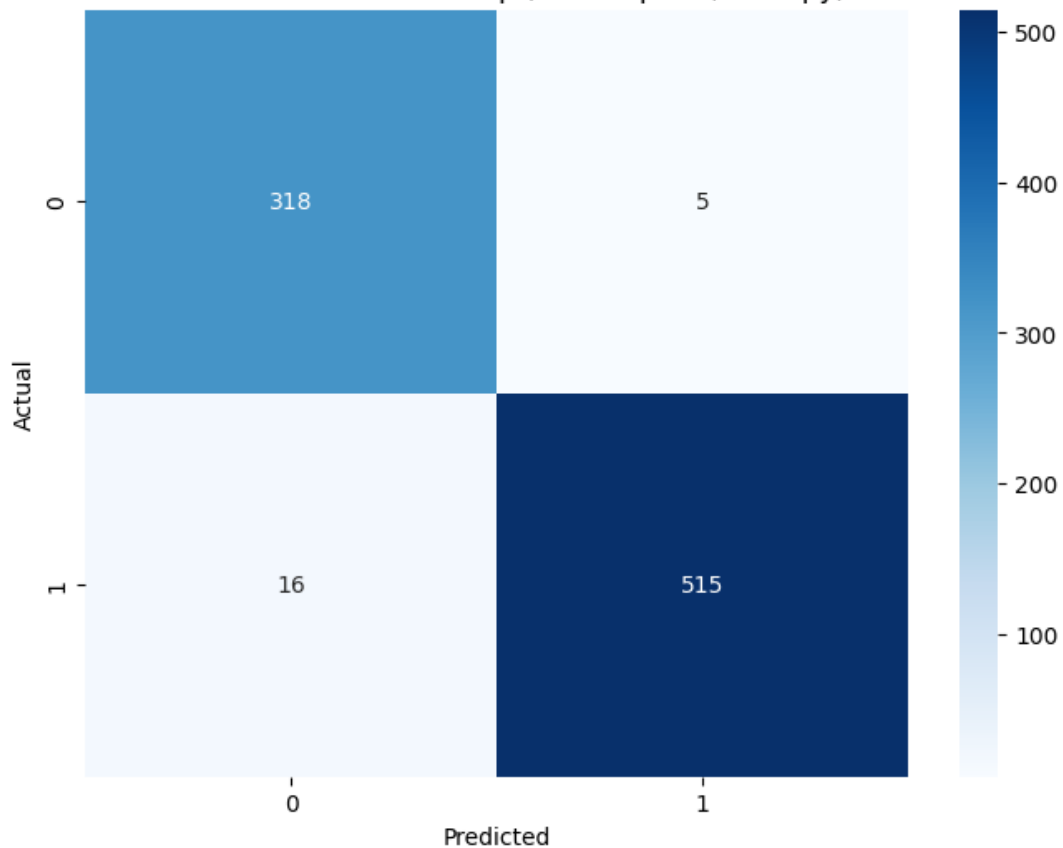
```
dt_entropy_5 = DecisionTreeClassifier(criterion='entropy', max_depth=5, random_state=42)
dt_entropy_5.fit(X_train, y_train)
y_pred_5 = dt_entropy_5.predict(X_test)
accuracy_5 = accuracy_score(y_test, y_pred_5)
print(f"Accuracy (Max Depth 5, Entropy): {accuracy_5:.4f}")
print("Confusion Matrix (Max Depth 5):")
print(confusion_matrix(y_test, y_pred_5))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_5), annot=True, fmt='d', cmap='Blues',
            xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 5, Entropy)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output:

```
Accuracy (Max Depth 5, Entropy): 0.9637
Confusion Matrix (Max Depth 5):
[[304 19]
 [12 519]]
```

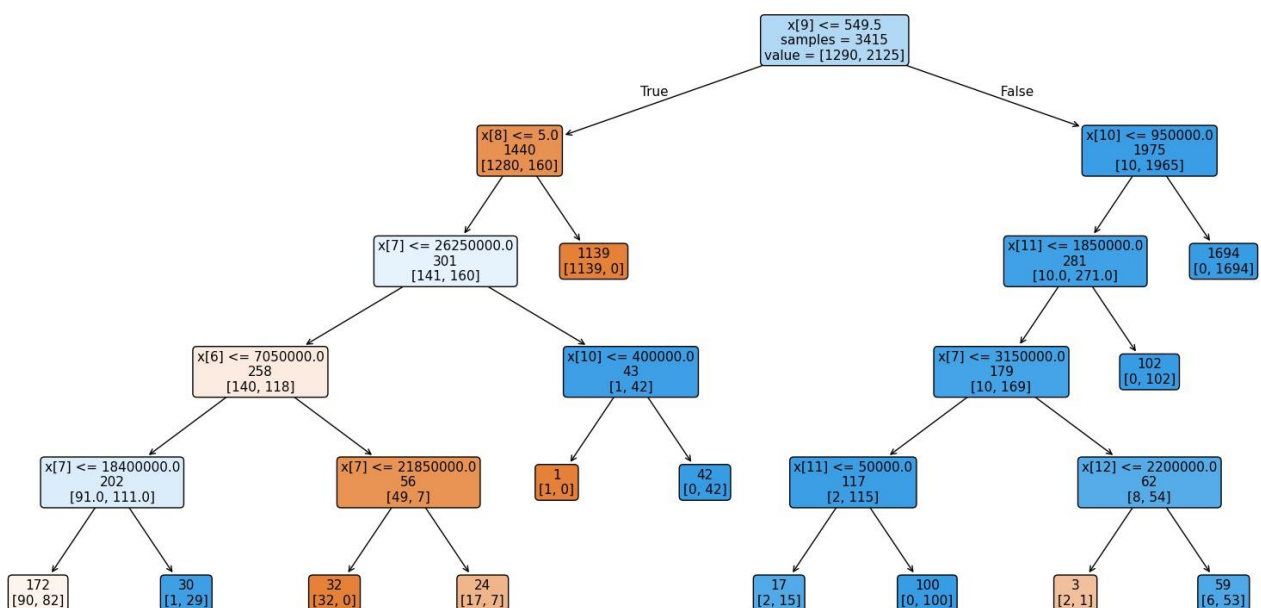
Confusion Matrix Heatmap (Max Depth 5, Entropy)



Plot the tree (Entropy - Max Depth 5)

```
plt.figure(figsize=(20, 10))
plot_tree(dt_entropy_5, filled=True, rounded=True, label='root', impurity=False,
          proportion=False)
plt.title("Decision Tree (Entropy) - Max Depth 5")
plt.show()
```

Decision Tree (Entropy) - Max Depth 5



Train Decision Tree with max_depth=7 (Entropy)

```

dt_entropy_7 = DecisionTreeClassifier(criterion='entropy', max_depth=7, random_state=42)
dt_entropy_7.fit(X_train, y_train)
y_pred_7 = dt_entropy_7.predict(X_test)
accuracy_7 = accuracy_score(y_test, y_pred_7)
print(f"Accuracy (Max Depth 7, Entropy): {accuracy_7:.4f}")
print("Confusion Matrix (Max Depth 7):")
print(confusion_matrix(y_test, y_pred_7))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_7), annot=True, fmt='d', cmap='Blues',
            xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 7, Entropy)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

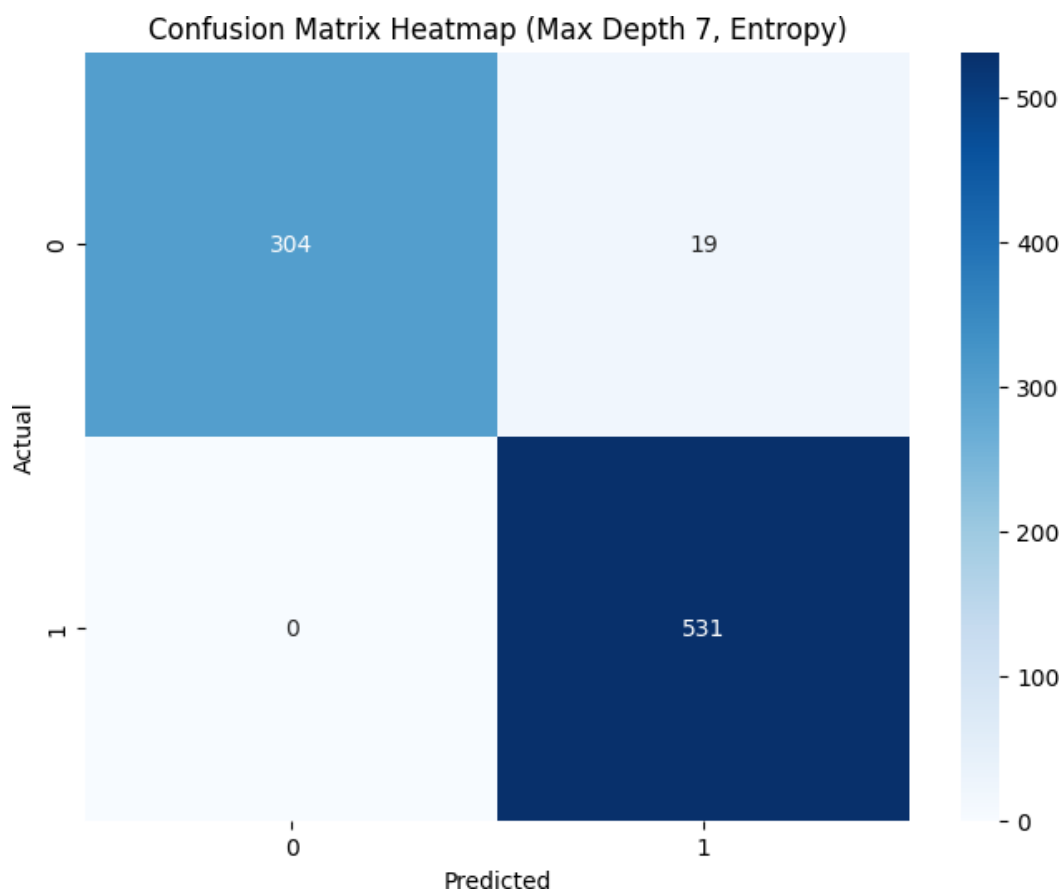
```

Output:

```

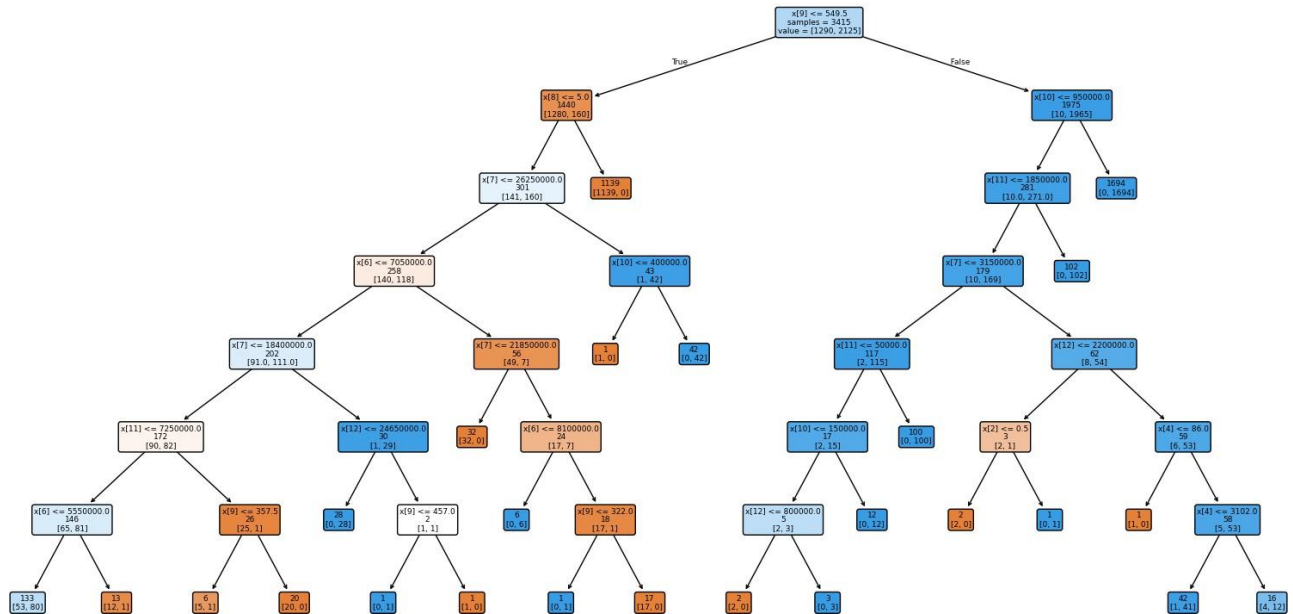
Accuracy (Max Depth 7, Entropy): 0.9766
Confusion Matrix (Max Depth 7):
[[312  11]
 [  9 522]]

```

**# Plot the tree (Entropy - Max Depth 7)**

```
plt.figure(figsize=(20, 10))
plot_tree(dt_entropy_7, filled=True, rounded=True, label='root', impurity=False,
          proportion=False)
plt.title("Decision Tree (Entropy) - Max Depth 7")
plt.show()
```

Decision Tree (Entropy) - Max Depth 7



Option B: Multiclass Classification (Obesity Dataset)

Step 1: Importing Libraries

Import necessary libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import LabelEncoder
print("Libraries imported successfully!")
```

Output:

```
Libraries imported successfully!
```

Step 2: Loading Data

Load the dataset

```
data = pd.read_csv('ObesityDataSet_raw_and_data_synthetic.csv')
print("Dataset loaded successfully")
```

Output:

Dataset loaded successfully

Check the shape of the dataset

```
print("Data Shape:")
print(data.shape)
```

Output:

Data Shape:
(2111, 17)

Display first 5 rows of the dataset

```
print("First 5 rows of the dataset:")
data.head()
```

Output:

First 5 rows of the dataset:

Output:

	Gender	Age	Height	Weight	family_history_with_overweight	FA	FC	NC	CAEC	SMO	CH	SC	FA	TU	CALC	MTRANS	NObesidad
0	Female	21	1.62	64.0	yes	no	2.0	3.0	Sometimes	no	2.0	no	0.0	1.0	no	Public_Transportation	Normal_Weight
1	Female	21	1.52	56.0	yes	no	3.0	3.0	Sometimes	yes	3.0	yes	3.0	0.0	Sometimes	Public_Transportation	Normal_Weight
2	Male	23	1.80	77.0	yes	no	2.0	3.0	Sometimes	no	2.0	no	2.0	1.0	Frequently	Public_Transportation	Normal_Weight
3	Male	27	1.80	87.0	no	no	3.0	3.0	Sometimes	no	2.0	no	2.0	0.0	Frequently	Walking	Overweight_Level_I
4	Male	22	1.78	89.8	no	no	2.0	1.0	Sometimes	no	2.0	no	0.0	0.0	Sometimes	Public_Transportation	Overweight_Level_II

Display last 5 rows of the dataset

```
print("Last 5 rows of the dataset:")
data.tail()
```

Output:

Last 5 rows of the dataset:

Output:

	Gender	Age	Height	Weight	family_history_with_overweight	FA	FC	NC	CAEC	SMO	CH2O	SC	FAF	TUE	CALC	MTRANS	NObesidad
21	Female	20.9768	1.7107	131.4085	yes	yes	3.0	3.0	Sometim	no	1.7281	no	1.6762	0.9062	Sometim	Public_Transportatio	Obesity_Type_III
210	Female	21.9829	1.7485	133.7429	yes	yes	3.0	3.0	Sometim	no	2.00513	no	1.34139	0.5992	Sometim	Public_Transportatio	Obesity_Type_III
21	Female	22.5240	1.7522	133.6893	yes	yes	3.0	3.0	Sometim	no	2.0541	no	1.4142	0.6462	Sometim	Public_Transportatio	Obesity_Type_III
210	Female	24.3619	1.7394	133.3466	yes	yes	3.0	3.0	Sometim	no	2.85233	no	1.13910	0.5860	Sometim	Public_Transportatio	Obesity_Type_III
21	Female	23.6647	1.7388	133.4726	yes	yes	3.0	3.0	Sometim	no	2.8635	no	1.0264	0.7141	Sometim	Public_Transportatio	Obesity_Type_III

Step 3: Preprocessing

```
print("Statistical summary of the dataset:")
data.describe()
```

Output:

Statistical summary of the dataset:

Output:

	Age	Height	Weight	PCVC	NCP	CH2O	FAF	TUE
count	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000
mean	24.312600	1.701677	86.586058	2.419043	2.685628	2.008011	1.010298	0.657866
std	6.345968	0.093305	26.191172	0.533927	0.778039	0.612953	0.850592	0.608927
min	14.000000	1.450000	39.000000	1.000000	1.000000	1.000000	0.000000	0.000000
25%	19.947192	1.630000	65.473343	2.000000	2.658738	1.584812	0.124505	0.000000
50%	22.777890	1.700499	83.000000	2.385502	3.000000	2.000000	1.000000	0.625350
75%	26.000000	1.768464	107.430682	3.000000	3.000000	2.477420	1.666678	1.000000
max	61.000000	1.980000	173.000000	3.000000	4.000000	3.000000	3.000000	2.000000

Initialize LabelEncoder

```
le = LabelEncoder()
target_names = []
print("LabelEncoder initialized")
```

Output:

LabelEncoder initialized

Apply encoding to categorical columns

```
for col in data.columns:
    if data[col].dtype == 'object':
        data[col] = le.fit_transform(data[col])
    if col == 'NObeyesdad':
        target_names = list(le.classes_)
print("Categorical variables encoded")
```

Output:

Categorical variables encoded

Split data into training and testing sets

```
X = data.drop('NObeyesdad', axis=1)
y = data['NObeyesdad']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("Data split complete")
```

Output:

Data split complete

Step 4: Training (Gini)

Initialize and train Decision Tree with Gini impurity

```
model = DecisionTreeClassifier(criterion='gini')
model.fit(X_train, y_train)
print("Model trained using Gini impurity")
```

Output:

```
Model trained using Gini impurity
```

Evaluate Gini model

```
y_pred = model.predict(X_test)
print("Accuracy (Gini, Full Tree):", accuracy_score(y_test, y_pred))
```

Output:

```
Accuracy (Gini, Full Tree): 0.9433
```

Compute confusion matrix for Gini

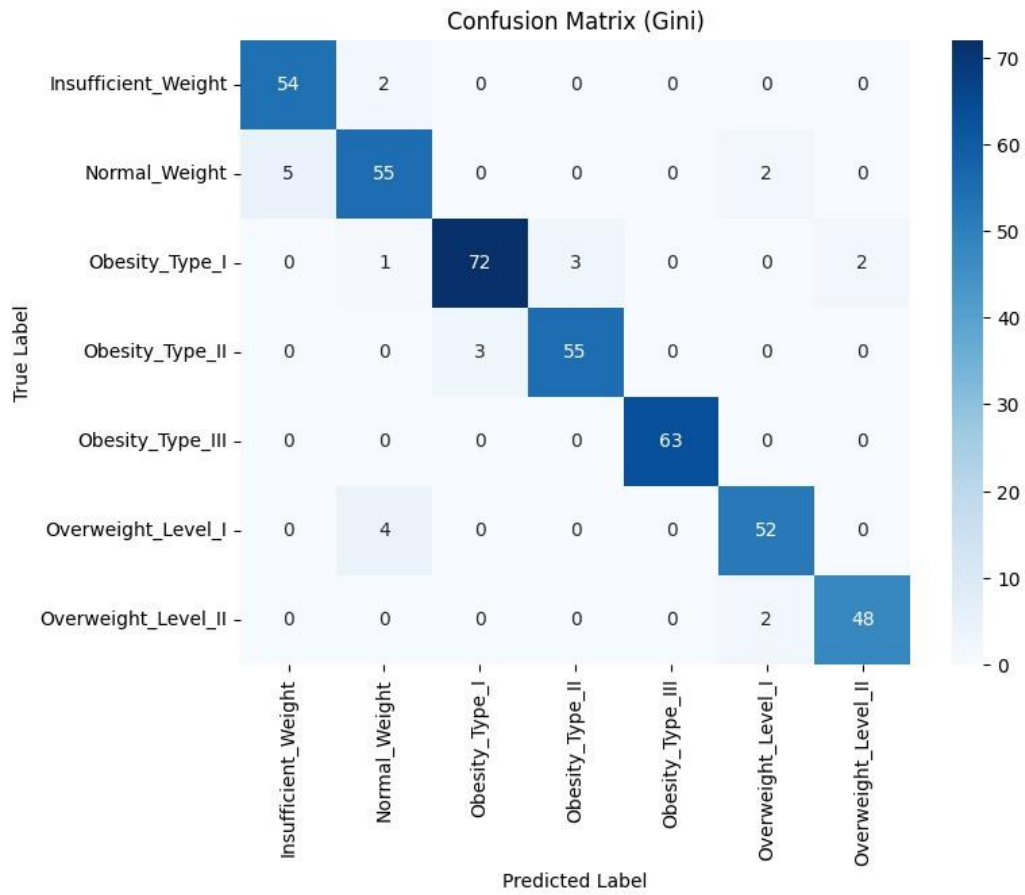
```
cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix shape:", cm.shape)
```

Output:

```
Confusion matrix shape: (7, 7)
```

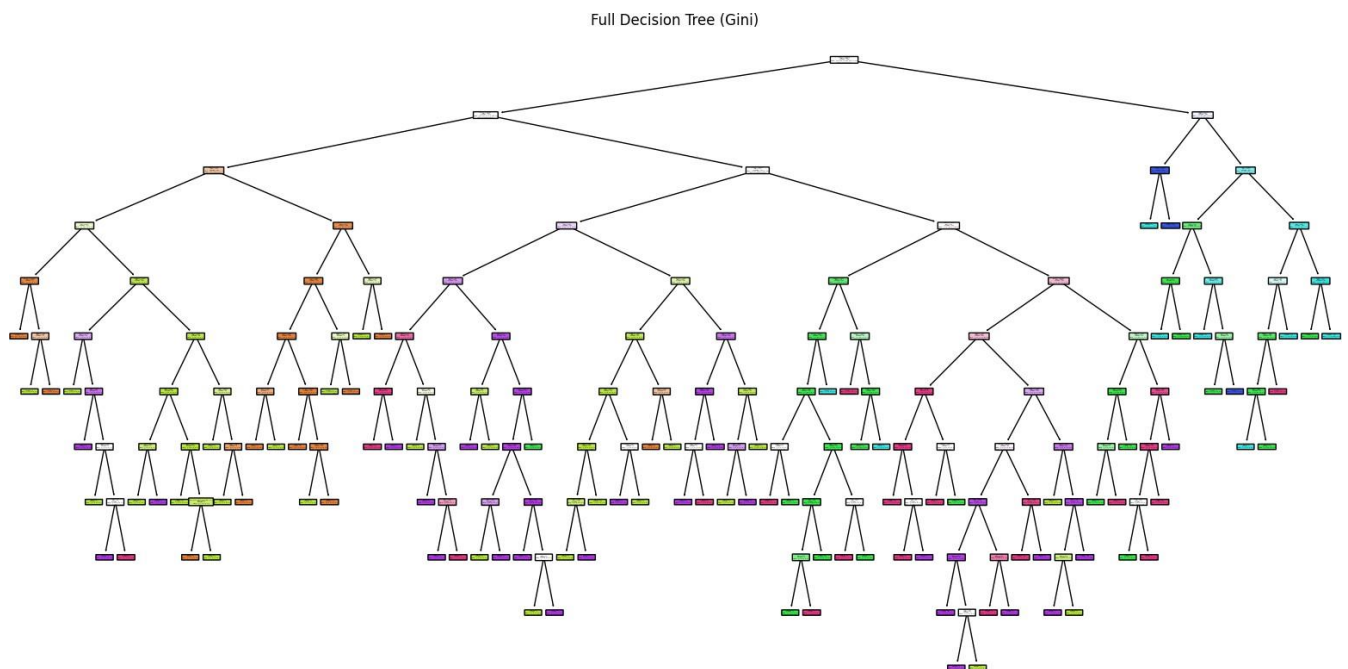
Plot confusion matrix (Gini)

```
print("Displaying plot...")
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=target_names,
            yticklabels=target_names)
plt.title('Confusion Matrix (Gini)')
plt.show()
```



Visualize the Gini Decision Tree

```
print("Plotting the Decision Tree (Gini)...")
plt.figure(figsize=(20,10))
plot_tree(model, feature_names=X.columns, class_names=target_names, filled=True,
          rounded=True)
plt.show()
```



Step 5: Training (Entropy)

Train and evaluate Decision Tree with Entropy

```
model_entropy = DecisionTreeClassifier(criterion='entropy')
model_entropy.fit(X_train, y_train)
y_pred_entropy = model_entropy.predict(X_test)
print("Accuracy (Entropy, Full Tree):", accuracy_score(y_test, y_pred_entropy))
```

Output:

```
Accuracy (Entropy, Full Tree): 0.9598
```

Compute confusion matrix for Entropy

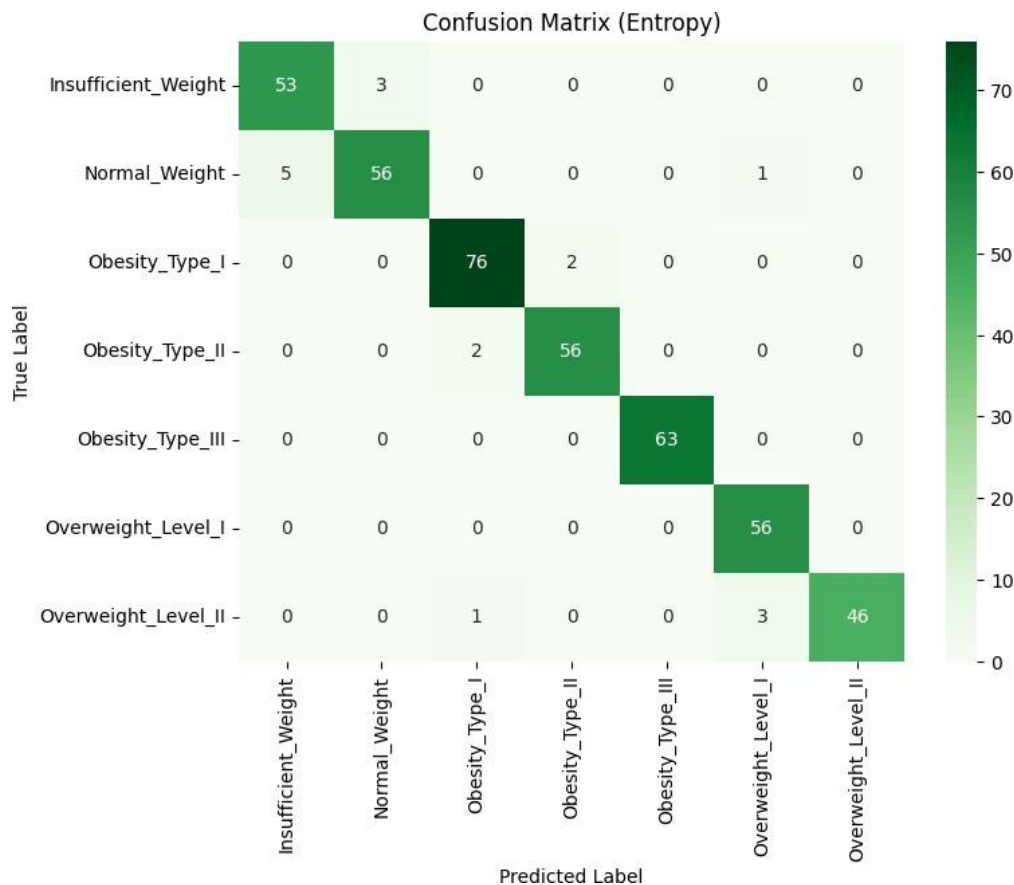
```
cm_entropy = confusion_matrix(y_test, y_pred_entropy)
print("Confusion matrix shape:", cm_entropy.shape)
```

Output:

```
Confusion matrix shape: (7, 7)
```

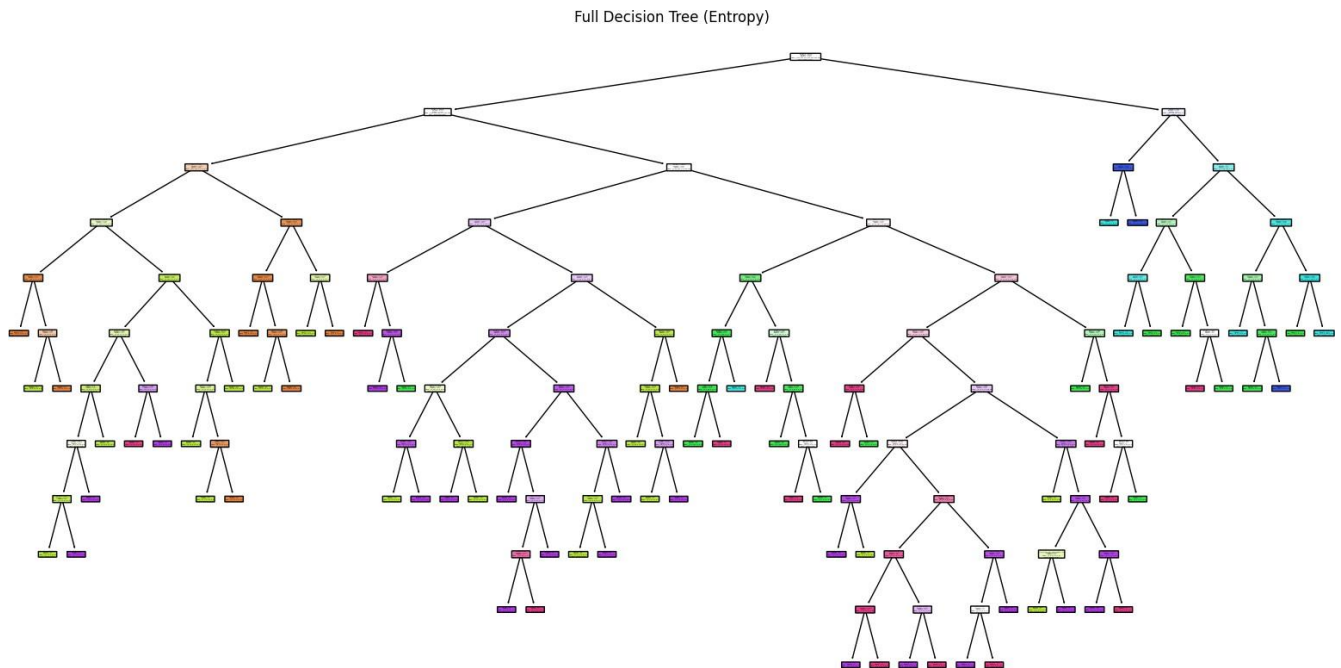
Plot confusion matrix (Entropy)

```
print("Displaying plot...")
plt.figure(figsize=(8,6))
sns.heatmap(cm_entropy, annot=True, fmt='d', cmap='Greens', xticklabels=target_names,
            yticklabels=target_names)
plt.show()
```



Visualize the Entropy Decision Tree

```
print("Plotting the Decision Tree (Entropy)...")
plot_tree(model_entropy, feature_names=X.columns, class_names=target_names, filled=True,
          rounded=True)
plt.show()
```



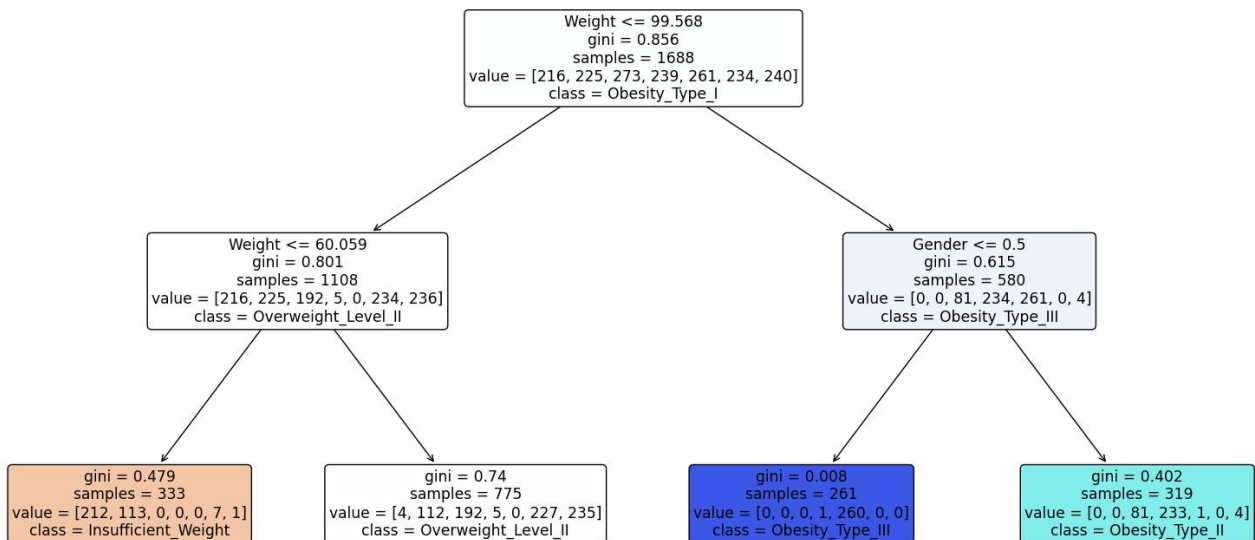
Step 6: Model Evaluation (Depth)

Analyze Decision Tree with Max Depth = 2

```
d = 2
clf = DecisionTreeClassifier(max_depth=d, criterion='gini', random_state=42)
clf.fit(X_train, y_train)
acc = accuracy_score(y_test, clf.predict(X_test))
print(f"Max Depth: {d}, Accuracy: {acc:.4f}")
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=X.columns, class_names=target_names, filled=True,
          rounded=True)
plt.title(f"Decision Tree (Max Depth = {d})")
plt.show()
```

Output:

Max Depth: 2, Accuracy: 0.5432



Analyze Decision Tree with Max Depth = 4

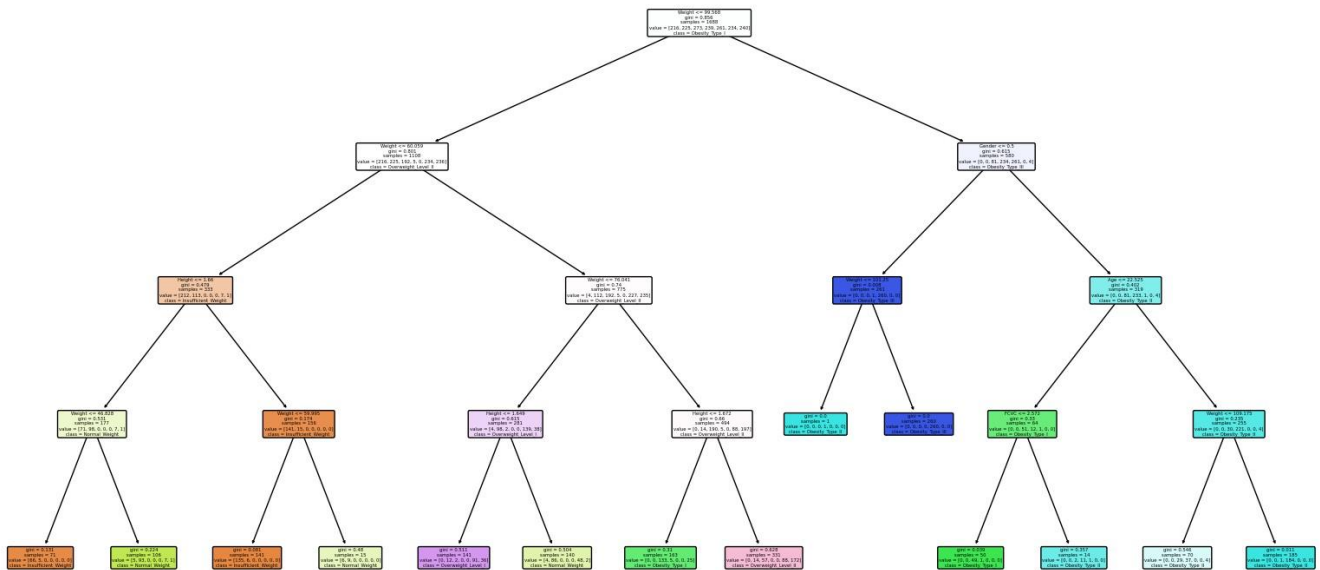
```
d = 4
clf = DecisionTreeClassifier(max_depth=d, criterion='gini', random_state=42)
clf.fit(X_train, y_train)
acc = accuracy_score(y_test, clf.predict(X_test))
print(f"Max Depth: {d}, Accuracy: {acc:.4f}")
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=X.columns, class_names=target_names, filled=True,
          rounded=True)
plt.title(f"Decision Tree (Max Depth = {d})")
plt.show()
```

Output:

Max Depth: 4, Accuracy: 0.9433

Decision Trees - Practical Implementation

Decision Tree (Max Depth = 4)



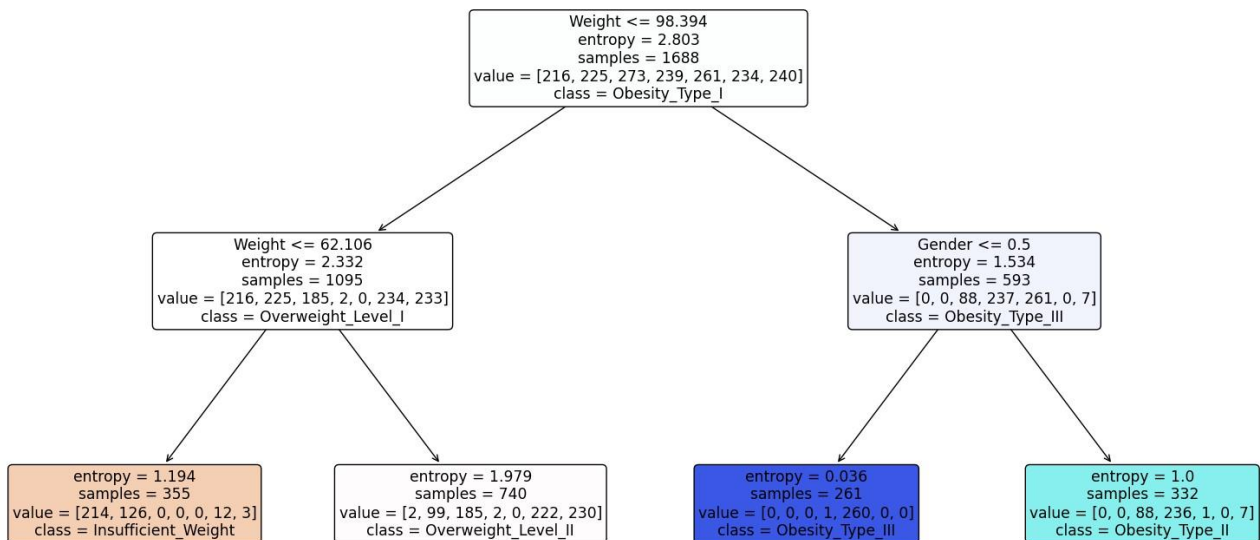
Analyze Decision Tree with Max Depth = 2 (Entropy)

```
d = 2
clf = DecisionTreeClassifier(max_depth=d, criterion='entropy', random_state=42)
clf.fit(X_train, y_train)
acc = accuracy_score(y_test, clf.predict(X_test))
print(f"Max Depth (Entropy): {d}, Accuracy: {acc:.4f}")
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=X.columns, class_names=target_names, filled=True,
          rounded=True)
plt.title(f"Decision Tree (Max Depth = {d}, Entropy)")
plt.show()
```

Output:

Max Depth (Entropy): 2, Accuracy: 0.9456

Decision Trees - Practical Implementation
Decision Tree (Max Depth = 2, Entropy)



Analyze Decision Tree with Max Depth = 4 (Entropy)

```
d = 4
clf = DecisionTreeClassifier(max_depth=d, criterion='entropy', random_state=42)
clf.fit(X_train, y_train)
acc = accuracy_score(y_test, clf.predict(X_test))
print(f"Max Depth (Entropy): {d}, Accuracy: {acc:.4f}")
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=X.columns, class_names=target_names, filled=True,
          rounded=True)
plt.title(f"Decision Tree (Max Depth = {d}, Entropy)")
plt.show()
```

Output:

Max Depth (Entropy): 4, Accuracy: 0.9598

Decision Tree (Max Depth = 4, Entropy)

